

**UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA**



Actualización del núcleo tecnológico de EDDIE

Estudiante: Nicolás Alonso Muñoz Reyes
Carrera: Ingeniería Civil en Informática
Profesor guía: Roberto González Ibáñez
Profesor Asignatura: Mario Inostroza Ponta

28 de agosto de 2026

TABLAS DE CONTENIDOS

1. Descripción del problema.....	4
1.1. Motivación y contexto.....	4
1.2. Enunciado del problema.....	4
2. Análisis de la solución.....	5
2.1. Estado del arte.....	5
2.1.1. Evolución del proyecto EDDIE.....	5
2.1.2. El problema de la modernización en software de investigación.....	5
2.1.3. Referentes de lectura aumentada y orquestación de sensores.....	6
2.2. Enfoques de solución.....	6
Enfoque 1: Corrección incremental del código en C#.....	6
Enfoque 2: Re-implementación completa con arquitectura de microservicios.....	7
Enfoque 3: Reconstrucción del núcleo mediante un contrato único de plugins en Python.....	7
2.3. Justificación del enfoque seleccionado.....	7
3. Descripción de la solución propuesta.....	7
3.1. Propósito de la solución.....	7
3.2. Características de la solución.....	8
3.3. Alcances y limitaciones de la solución.....	9
3.4. Evaluación de la solución.....	9
3.4.1. Pruebas de Software.....	9
3.4.2. Verificación y validación.....	10
4. Objetivos del proyecto.....	10
4.1. Objetivo general.....	10
4.2. Objetivos específicos.....	10
5. Metodología, herramientas y ambiente de desarrollo.....	11
5.1. Metodología a usar.....	11
5.2. Herramientas de desarrollo.....	11
5.3. Ambiente de desarrollo.....	12
6. Plan de trabajo.....	12
7. Glosario.....	13
8. Referencias.....	14
9. Anexo.....	16

ÍNDICE DE TABLAS

Tabla 2.1: Comparación de sistemas.....	16
Tabla 3.1: Extracto de errores encontrados en EDDIE.....	17
Tabla 3.2: Conteo por categoría de los 67 errores encontrados en EDDIE.....	17
Tabla 3.3: Ejemplo de matriz de trazabilidad de los errores de EDDIE.....	18
Tabla 6.1: Plan de trabajo.....	22

ÍNDICE DE FIGURAS

Figura 3.1: Arquitectura original monolítica de EDDIE.....	19
Figura 3.2: Arquitectura objetivo modular de EDDIE.....	20

1. Descripción del problema

1.1. Motivación y contexto

La lectura constituye un proceso cognitivo fundamental para el aprendizaje y la construcción de conocimiento, el cual ha experimentado una transformación sustancial con la digitalización de la información. En el ámbito de la Interacción Humano-Información (HII), resulta de especial interés analizar cómo las personas interactúan con distintos soportes de lectura y de qué manera estas modalidades impactan en factores como la comprensión lectora, la retención de ideas y la carga cognitiva. A pesar de la masificación de los dispositivos electrónicos, la evidencia empírica muestra que el material impreso conserva vigencia y sigue siendo preferido para tareas de lectura profunda (Saleh & Mashhur, 2015; Singer & Alexander, 2016, como se citaron en Santibáñez Espinoza, 2020).

Para avanzar en el estudio sistemático de estos fenómenos, la comunidad científica requiere de herramientas y metodologías experimentales rigurosas. La investigación en este campo depende de entornos que integren adecuadamente componentes de hardware, software y diseños experimentales capaces de capturar y evaluar, de manera precisa y no intrusiva, las dinámicas de interacción del lector durante el proceso de lectura. Dentro de las aproximaciones tecnológicas en esta área, una de las líneas menos exploradas y estudiadas es la lectura aumentada aplicada sobre papel físico. En este contexto, la lectura aumentada no se refiere a la superposición de animaciones recreativas o elementos tridimensionales lúdicos, sino al enriquecimiento del material impreso mediante prestaciones funcionales equivalentes a las del formato digital (tales como búsquedas en línea de términos dentro del texto, traducción instantánea, consulta de definiciones en contexto, despliegue de contenido complementario y sincronización bidireccional de anotaciones entre el documento físico y su versión digital).

Las herramientas capaces de soportar este tipo de experiencias integradas son escasas tanto en la literatura como en la práctica experimental. En el contexto local de investigación del Departamento de Ingeniería Informática de la Universidad de Santiago de Chile (USACH), y bajo la línea de trabajo del grupo InTeractiOn, nació la plataforma EDDIE (Empowering Digital Paper with Interactive Enhancements), concebida precisamente como un instrumento de investigación para habilitar experiencias de lectura aumentada no obstructiva sobre documentos impresos (Carrasco Madriaga, 2017). A lo largo de los años, EDDIE ha experimentado múltiples fases de desarrollo e integración incremental a través de aportes de distintos memoristas e investigadores de la unidad académica, quienes buscaron incorporar capacidades de procesamiento visual y seguimiento (Ibaceta Jaña, 2023).

Sin embargo, debido al tiempo transcurrido desde su concepción original y a la acumulación de evoluciones heterogéneas desarrolladas en diferentes etapas, la herramienta actualmente presenta limitaciones y desactualizaciones operativas. Estas condiciones impiden que EDDIE pueda ser utilizada de manera plena, estable y confiable en estudios experimentales contemporáneos sobre comprensión y comportamiento lector, limitando su potencial como plataforma de medición científica.

1.2. Enunciado del problema

El problema central del proyecto EDDIE no radica en el volumen total de errores documentados, sino en un riesgo arquitectónico específico sobre una base de aproximadamente 45.000 líneas de código distribuidas en 26 proyectos. De los 67 problemas registrados por Ibaceta (2023), su reclasificación por causa (Tabla 3.1 del Anexo) muestra que 16 (23%) corresponden a la infraestructura técnica de carga y despliegue,

mientras que el resto se distribuye entre errores de práctica de código, hardware/sensores, servicios externos, UI/idioma, infraestructura de laboratorio y documentación. Aunque los 4 errores de carga por reflexión representan sólo una fracción de esta infraestructura técnica, tienen un impacto desproporcionado: cada uno impide el arranque completo de un módulo entero, y su causa raíz constituye un riesgo estructural recurrente: cualquier desarrollador que integre un nuevo plugin puede reproducir el mismo tipo de falla, dado que no existe una única interfaz validada que lo prevenga. Ante este escenario, ¿En qué medida la reconstrucción del núcleo de EDDIE bajo una arquitectura de plugins con contrato único permite reducir el número de errores estructurales documentados y habilitar su uso como instrumento de medición científica confiable?

2. Análisis de la solución

2.1. Estado del arte

2.1.1. Evolución del proyecto EDDIE

El sistema EDDIE se enfoca en la lectura aumentada no obstructiva, buscando unificar las ventajas del papel con las capacidades digitales. Su desarrollo comenzó con sistemas de realidad aumentada móvil (Carrasco Madriaga, 2017) y evolucionó hacia proyecciones directas sobre papel (Contreras Velásquez, 2019). Investigaciones posteriores integraron búsqueda web (Montanares Mundaca, 2019), consistencia bidireccional (Pezoa Riquelme, 2020) y rastreo ocular (Estay Vera, 2020).

Este crecimiento ha evidenciado una asimetría funcional persistente, mientras el papel permite manipulación natural, el digital ofrece automatización (Guimbretière, 2003, como se citó en Carrasco Madriaga, 2017). Además, estudios en psicología cognitiva demuestran que el material impreso sigue siendo preferido por su eficacia en la retención de ideas (Saleh & Mashhur, 2015; Singer & Alexander, 2016, como se citaron en Santibáñez Espinoza, 2020). No obstante, evaluaciones mediante la metodología MECIVA PS documentaron 67 problemas de calidad en el sistema actual (Ibaceta Jaña, 2023), reclasificados por causa en la sección 1.2.

2.1.2. El problema de la modernización en software de investigación

En este trabajo se entiende por software científico aquel desarrollado ad hoc para soportar una tarea de investigación específica, como la captura, procesamiento o análisis de datos experimentales, en contraposición al software de propósito general que un laboratorio pueda utilizar como herramienta auxiliar (sistemas operativos, editores de texto, suites ofimáticas). Bajo esta definición, EDDIE y MO son software científico, mientras que el sistema operativo o el editor de código empleados durante su desarrollo no lo son, dado que no fueron construidos para responder a un problema de investigación particular.

Un desafío documentado en el desarrollo de este tipo de software es que decisiones técnicas localmente óptimas (las que son tomadas por un investigador para resolver el problema inmediato de su propia tesis) tienden a resultar en un sistema globalmente inconsistente cuando se acumulan a través de múltiples autores sin una arquitectura común (Ernst et al., 2026). Este fenómeno ha sido documentado en sistemas de mayor escala, como plataformas de bioinformática y biología computacional, donde la falta de coordinación entre grupos de desarrollo genera herramientas difíciles de integrar entre sí (Orozco Solano, 2017). La literatura converge en que la solución recurrente a este problema es la adopción de arquitecturas modulares con interfaces estandarizadas que separan la lógica de

investigación de la infraestructura técnica subyacente (Bustio Martínez et al., 2013; Glatard et al., 2017). En el dominio específico de los science gateways (plataformas que integran instrumentos y flujos de trabajo científicos), esta separación se implementa habitualmente mediante una capa de abstracción de hardware y un catálogo de plugins cargados dinámicamente a través de un único mecanismo, lo que permite aislar el fallo de un componente sin comprometer el resto del sistema (Glatard et al., 2017). Esta misma arquitectura de núcleo liviano con contratos de interfaz bien definidos es la que permite minimizar el tiempo dedicado a resolver problemas comunes de integración, concentrando el esfuerzo de desarrollo en la implementación de los algoritmos propios de cada dominio de investigación (Bustio Martínez et al., 2013).

2.1.3. Referentes de realidad aumentada y orquestación de sensores

EDDIE se distingue de sistemas como DigitalDesk (Wellner, 1993), EnhancedDesk (Koike et al., 2001), AR-PaperSync (Miah & Kong, 2024) y Enlight (Wander, s.f.) por integrar simultáneamente proyección sobre papel, eye-tracking y consistencia bidireccional. Para gestionar tal heterogeneidad se requiere de un modelo de orquestación donde un controlador central coordina el flujo mediante patrones como el adaptador (Megargel et al., 2021). Actualmente, se recomienda el uso de arquitecturas basadas en plugins con núcleos livianos que garanticen la extensibilidad (Bustio Martínez et al., 2013).

El antecedente es Multimodal Observer (MO), un sistema del mismo laboratorio InTeractiOn que enfrentó una crisis de deuda técnica estructuralmente idéntica: las sucesivas versiones de su núcleo, desarrolladas por distintos tesisistas para incorporar nuevos módulos, generaron incompatibilidades que dejaron la mayoría de sus plugins de visualización y análisis inoperativos en las versiones más recientes. La solución adoptada fue una reimplementación completa del núcleo en Python con una arquitectura de plugins desacoplada bajo un contrato único, la cual demostró ser viable en el contexto académico y produjo un sistema validado con métricas objetivas de calidad de código, rendimiento y aceptación (Figuerola Contreras, 2025).

2.2. Enfoques de solución

Para abordar la deuda técnica acumulada y la inestabilidad arquitectural del sistema EDDIE, se han ideado y validado tres enfoques de solución, considerando su viabilidad técnica y su impacto en la investigación del laboratorio.

Enfoque 1: Carga dinámica

Este enfoque consiste en reemplazar la carga estática de plugins por una carga dinámica (Assembly.LoadFrom, AppDomain.Load) que elimina las rutas hardcodeadas y los caracteres especiales de escape de caracteres, sin cambiar el lenguaje de programación. Esto reduce el riesgo de errores puntuales de dependencias, evita la duplicación de código (reducción de la deuda técnica total), dejando intactas las 40.000 líneas de lógica de negocio, búsqueda y rastreo ocular.

- Ventajas: preserva el código existente, presenta una curva de aprendizaje mínima para el equipo (ya familiarizado con .NET) y resuelve directamente los 16 errores de infraestructura de carga con una inversión considerablemente menor que una reimplementación completa.
- Limitaciones: no existe en el laboratorio un antecedente empírico de esta modernización sobre un sistema similar, mientras que la reimplementación en Python cuenta con el precedente validado de MO (Figuerola Contreras, 2025).

Persiste además la falta de validación de los módulos frente a sensores físicos reales (Ibaceta Jaña, 2023), lo que perpetúa la asimetría funcional entre medios físicos y digitales (Guimbretiére, 2003, como se citó en Carrasco Madriaga, 2017), independientemente del lenguaje elegido.

Enfoque 2: Re-implementación completa con arquitectura de microservicios

Reconstruye el sistema bajo un paradigma de servicios independientes (consistencia, rastreo ocular, reconocimiento de texto) comunicados mediante APIs estandarizadas, donde cada uno de los cinco tipos de dispositivo de EDDIE operaría como un proceso de red autónomo.

- Ventajas: desacoplamiento total, con cada componente escalable y actualizable de forma autónoma.
- Limitaciones: complejidad de infraestructura excesiva para un contexto académico. Gestionar comunicaciones de red y sincronizar en tiempo real procesos de distinta frecuencia (eye-tracking versus procesamiento de texto) requeriría una inversión de tiempo desproporcionada en soporte técnico.

Enfoque 3: Reconstrucción del núcleo mediante un contrato único de plugins en Python

Inspirado en la reimplementación exitosa de MO, reemplaza los tres mecanismos heterogéneos de carga por reflexión por un núcleo orquestador único (eddie-core) que gestiona todos los dispositivos mediante un contrato de interfaz común (PluginBase), complementado con una capa de abstracción de hardware que conmuta entre implementaciones reales y simuladas.

- Ventajas: Ataca la causa raíz identificada en la motivación, unificando en un solo mecanismo los tres orígenes de carga por reflexión responsables de 16 de los 67 errores documentados (categoría "infraestructura técnica de carga y despliegue", Tabla 3.2). Aprovecha el ecosistema Python de visión computacional y procesamiento de señales, con antecedente empírico validado en el mismo laboratorio para errores de integración análogos (Figueroa Contreras, 2025).
- Limitaciones: exige que el equipo asuma una curva de aprendizaje por el cambio de lenguaje, y requiere una definición rigurosa de los contratos de interfaz de plugin para evitar reproducir el mismo problema de heterogeneidad que se busca resolver.

2.3. Justificación del enfoque seleccionado

El enfoque seleccionado es el tercero. La decisión no se sostiene en que sea el único capaz de resolver la causa raíz de la deuda técnica de carga (el Enfoque 1 resuelve la misma cantidad de errores con una inversión de reescritura menor), sino en el antecedente empírico validado de MO bajo un problema estructuralmente análogo dentro del mismo laboratorio, y en la oportunidad de modernizar simultáneamente el ecosistema de visión computacional en una misma iteración. El Enfoque 2 se descarta por la sobrecarga de infraestructura que impone gestionar servicios de red independientes para sensores de distinta frecuencia, incompatible con el plazo de un proyecto de titulación.

3. Descripción de la solución propuesta

3.1. Propósito de la solución

El propósito de esta modernización es transformar el actual ecosistema de prototipos funcionales en una herramienta de investigación científica, capaz de cumplir con los

estándares de validez y confiabilidad del ámbito académico. Se busca reducir la deuda técnica acumulada durante años de crecimiento fragmentado, subsanando los problemas de inestabilidad detectados en módulos anteriores, y garantizar la validación del sistema en contextos reales de operación, permitiendo que herramientas críticas como el seguimiento ocular y la consistencia bidireccional funcionen correctamente con el hardware físico para el que fueron diseñadas. Asimismo, una base tecnológica estandarizada y robusta busca asegurar la escalabilidad y la transferencia de conocimiento, facilitando la incorporación de nuevos investigadores. En última instancia, la solución pretende consolidar un soporte avanzado para la investigación en HII, con una plataforma estable para estudios sobre comportamiento atencional, aprendizaje y comprensión lectora en entornos aumentados.

Más allá de reducir los errores de infraestructura de carga, esta reconstrucción tiene una implicancia directa sobre la validez experimental de EDDIE. La tabla de errores de Ibaceta (2023) (Tabla 3.1, Anexo) registra fallas no resueltas en la conexión con hardware físico, como la inestabilidad de la cámara digital (E16, solución parcial) y la imposibilidad de detectar ciertas áreas de la proyección mediante el eyetracker (E49, sin solución). Mientras estas fallas persisten, no es posible distinguir si una irregularidad en la mirada del lector responde a un comportamiento atencional genuino o a una falla del sensor, lo que invalida en la práctica el uso de EDDIE como instrumento de medición científica. La nueva arquitectura aborda esto unificando la comunicación con eyetrackers bajo la interfaz IEyeTracker y mediante la capa DeviceFactory, que permite validar el sistema con dispositivos simulados antes de una sesión real, reduciendo la probabilidad de confundir una falla técnica con una señal experimental válida.

3.2. Características de la solución

La propuesta reconstruye el núcleo tecnológico de EDDIE, actualmente ~45.000 líneas en 26 proyectos .csproj bajo una arquitectura monolítica: un coordinador central (InteractionCoordinator) acoplado a la interfaz WinForms, que carga sus extensiones mediante tres mecanismos heterogéneos de reflexión, origen de 16 de los 67 errores diagnosticados (Figura 3.1, Tabla 3.2, Anexo).

El núcleo propuesto opera dentro de un único proceso Python, compuesto por el OrchestratorCore, con registro dinámico en memoria y un ciclo estandarizado de tres fases (inicialización, ejecución, liberación), y el PluginLoader, que instancia los plugins declarados en un archivo de configuración como objetos en el mismo espacio de memoria. La comunicación orquestador-plugin ocurre mediante llamadas a método directas sobre la interfaz PluginBase, sin red, serialización ni IPC. Esto prioriza simplicidad sobre aislamiento de fallos. Como mitigación, cada llamada se ejecuta dentro de un bloque de manejo de excepciones gestionado centralmente por el OrchestratorCore, de modo que una falla en un plugin queda aislada y registrada sin detener el sistema.

Se definen cinco contratos sobre PluginBase (IImageProcessor, IEyeTracker, IGesturePlugin, IWebSearchProvider, IConsistencyProvider), dado el bajo costo de especificar una interfaz frente a implementarla. El MVP implementa y valida con hardware real cuatro de ellos: IConsistencyProvider, IEyeTracker (GazePoint, TheEyeTribe), IImageProcessor e IGesturePlugin (cámaras digitales del laboratorio InTeractiOn). IWebSearchProvider queda solo a nivel de interfaz, sin implementación validada, por depender de APIs externas fuera del control del proyecto. El hardware físico se abstrae mediante DeviceFactory, que conmuta entre implementaciones reales y simuladas según variable de entorno (Figura 3.2, Anexo).

Se estima una base resultante de 8.000 a 10.000 líneas efectivas, tras eliminar las tres capas redundantes de carga por reflexión y sustituir las dependencias nativas de 32 bits por bibliotecas Python multiplataforma (OpenCV, PyMuPDF). La solución incluye documentación técnica del núcleo y de los cinco contratos de interfaz.

3.3. Alcances y limitaciones de la solución

La solución cubre la reconstrucción completa del núcleo orquestador, los cinco contratos de interfaz de plugin (sección 3.2) y la capa de abstracción de hardware (DeviceFactory) con conmutación real/simulado. La validación con hardware real se limita a los cuatro módulos del MVP (IConsistencyProvider, IEyeTracker sobre GazePoint/TheEyeTribe, IImageProcessor e IGesturePlugin) en el ambiente del laboratorio InTeractiOn. La implementación funcional de IWebSearchProvider (queda solo como interfaz), dashboard web, visor de proyección en navegador, CLI de validación y su canal de telemetría WebSocket, reconocimiento automático del libro físico y catálogo de lectura, eliminación de accesorios mediadores (mouse, lápices de color) y recalibración de visión computacional más allá de la mínima necesaria para el MVP. La migración de Emgu CV a OpenCV no es una traducción directa (ambas bibliotecas producen resultados numéricos distintos ante los mismos parámetros), por lo que los algoritmos de reconocimiento se deben recalibrar y validar en el nuevo entorno. Al ejecutarse todos los plugins como objetos en memoria dentro del proceso del OrchestratorCore, se elimina el overhead de red, pero el aislamiento de fallos depende del manejo de excepciones centralizado (sección 3.2). La proyección se renderiza en el mismo proceso mediante ventana nativa, el WebSocket queda reservado para telemetría externa, fuera de este alcance. Además se eliminan las dependencias nativas de 32 bits mejorando la portabilidad potencial del software, pero no está verificada el desarrollo es exclusivamente en Windows 11 y depende de SDKs de fabricante específicos de plataforma. La verificación en Linux/macOS queda fuera de alcance, y en cualquier caso esta mejora no implica portabilidad del sistema completo, atado a un montaje físico particular.

3.4. Evaluación de la solución

La evaluación seguirá un diseño experimental distribuido a lo largo de las 17 semanas del proyecto, el cual estará organizado en tres niveles progresivos que permitirán validar el sistema desde sus componentes individuales hasta su operación integrada en condiciones reales de investigación.

3.4.1. Pruebas de Software

Este primer nivel de evaluación mide la solidez arquitectónica del núcleo y los contratos de interfaz mediante tres métricas. Primero, la cobertura de pruebas unitarias e integración, medida como cobertura de líneas de código, con un umbral mínimo del 80%, replicando el criterio aplicado por Figueroa Contreras (2025) en el backend de MO (95,2% de líneas cubiertas). Segundo, en lugar de contrastar el número de errores de integración del nuevo núcleo contra el total de 67 problemas documentados (magnitudes no comparables, dado que la primera mide solo fallas de integración y la segunda agrega problemas de hardware, documentación y usabilidad), se reutilizan los casos de prueba ya documentados por Pezoa (2020) para consistencia, Contreras Velásquez (2019) para reconocimiento de texto e interacción gestual, y Estay (2020) para rastreo ocular, ejecutándose tanto en la versión actual de EDDIE como en el prototipo Python y registrando el resultado en una matriz de trazabilidad (Tabla 3.3) que clasifica cada caso como corregido, persistente o fuera de alcance, con evidencia asociada (test, log o nota de

alcance). Tercero, se mide la latencia de comunicación entre el orquestador y cada plugin, con un umbral derivado de la frecuencia real de los eyetrackers GazePoint y TheEyeTribe (~60 Hz, ~16,6 ms por muestra): la latencia interna no debe superar el 50% de ese presupuesto ($\leq 8,3$ ms).

Se incorporan además métricas por dominio para sostener que EDDIE produce mediciones científicamente válidas. En sincronización entre sensores, se mide el desfase temporal máximo (ms) entre eventos de distintos plugins que deberían estar alineados (por ejemplo, una fijación ocular y su resaltado de texto correspondiente), usando timestamps de referencia del OrchestratorCore, y la tasa de pérdida de eventos durante captura simultánea. En visión computacional, se evalúa precisión y recall del reconocimiento de texto contra un set de al menos 10 palabras de distinto largo (siguiendo la metodología de Ibaceta, 2023) y una tasa de error para el reconocimiento gestual, definida como proporción de gestos correctamente clasificados sobre el total ejecutado en un protocolo repetido.

3.4.2. Verificación y validación

Este nivel replicará la metodología MECIVA PS para cada módulo integrado bajo la nueva arquitectura (Ibaceta Jaña, 2023), clasificando previamente los requisitos en críticos (deben cumplirse en el 100% de las ejecuciones, por ejemplo la carga sin fallos de los tres plugins del MVP o un error espacial del rastreo ocular dentro de un margen predefinido, su incumplimiento se reporta como bloqueante) y secundarios (evaluados bajo la escala de cinco niveles descrita más abajo). Se evaluará la validez funcional, definida como el porcentaje de requisitos heredados que el módulo cumplirá correctamente, y la confiabilidad, medida mediante coeficiente de variación para variables numéricas continuas (ej. coordenadas de fijación ocular) o tasa de éxito para resultados binarios (ej. carga correcta de un plugin), sobre un conjunto fijo de al menos 10 ejecuciones por caso.

Finalmente, se realizarán las pruebas de aceptación en el ambiente operativo real del laboratorio InTeractiOn con el hardware físico disponible, aplicando a los requisitos secundarios la escala de evaluación de trabajos previos del laboratorio, desde "Completa" (85-100%) hasta "No aceptada" (0-14%) según el porcentaje de cumplimiento observado (Estay Vera, 2020). Se definirán escenarios que cubran la operación simultánea de módulos, la recuperación ante fallos parciales de sensores y la sincronización de datos entre componentes de frecuencias distintas.

4. Objetivos del proyecto

4.1. Objetivo general

Actualizar el núcleo tecnológico del proyecto EDDIE mediante el rediseño y reimplementación en Python e integración de los módulos de investigación existentes, para reducir la deuda técnica y asegurar la compatibilidad con sensores físicos del proyecto en un entorno de lectura aumentada no obstructiva.

4.2. Objetivos específicos

1. Diagnosticar la arquitectura actual de EDDIE en C# a partir de la reclasificación de errores de Ibaceta (2023), y evaluar mediante prototipos desechables en Python la factibilidad técnica de migrar los módulos existentes a las tecnologías propuestas.
2. Diseñar el núcleo orquestador de EDDIE, definiendo los contratos de interfaz que permitan la integración de los módulos previos como plugins desacoplados.

3. Implementar el núcleo orquestador en Python, junto con la capa de abstracción de sensores, integrando los módulos de consistencia bidireccional, reconocimiento de texto, interacción gestual y rastreo ocular bajo la nueva arquitectura.
4. Validar el sistema integrado mediante pruebas de software, verificación funcional y pruebas de aceptación en el ambiente operativo real del laboratorio InTeractiOn.

5. Metodología, herramientas y ambiente de desarrollo

5.1. Metodología a usar

Para la modernización del núcleo de EDDIE se emplea el ciclo PDCA (Plan-Do-Check-Act), complementado con Kanban para el seguimiento de tareas y Rapid Application Development (RAD) para el desarrollo iterativo mediante prototipos. Esta combinación replica el enfoque validado por Figueroa (2025) en la reimplementación de MO, dada la similitud estructural entre ambos proyectos. El ciclo PDCA estructura el trabajo en cuatro etapas, correspondientes a los objetivos específicos: Planificación (Plan), con el diagnóstico del estado actual de EDDIE y la identificación de los nudos críticos de incompatibilidad (OE1). Ejecución (Do), con el diseño e implementación del núcleo orquestador, la capa de abstracción de sensores y la integración de los módulos existentes como plugins mediante iteraciones sucesivas de prototipado bajo RAD (OE2 y OE3). Verificación (Check), con las pruebas de software, verificación funcional y pruebas de aceptación descritas en la sección de evaluación. Y Actuación (Act), con la incorporación de ajustes derivados de dichas pruebas y el cierre de la documentación técnica.

Un aspecto central de RAD es el uso de prototipos desechables (throwaway prototyping) durante las primeras semanas de la etapa de ejecución, en lugar de incorporar directamente cada funcionalidad migrada al código definitivo. Cada componente crítico se valida primero mediante un prototipo aislado y de bajo costo, cuyo único propósito es confirmar la factibilidad técnica de la migración a Python antes de invertir tiempo en su integración formal al orquestador. Este criterio, aplicado exitosamente por Figueroa (2025) en MO, permite detectar tempranamente bloqueos de compatibilidad y evita destinar semanas de desarrollo a funcionalidades que podrían requerir un enfoque alternativo. Kanban se utiliza para visualizar el flujo de tareas de cada módulo y fase del ciclo PDCA a lo largo de las 17 semanas del proyecto.

5.2. Herramientas de desarrollo

Las herramientas empleadas durante el desarrollo del proyecto son las siguientes:

- Python, lenguaje principal para la implementación del núcleo orquestador, la capa de abstracción de sensores y los plugins.
- OpenCV (binding para Python), utilizado para las funcionalidades de visión computacional que en la versión original de EDDIE dependían de Emgu CV.
- pytest, para el desarrollo de pruebas unitarias y de integración.
- SonarQube, para el análisis de calidad del código durante las pruebas de software.
- Git y GitHub, para el control de versiones y la gestión del repositorio del proyecto.
- Visual Studio Code, como entorno de desarrollo integrado.
- EDDIE (versión actual en C#), utilizado como referencia funcional durante el diagnóstico y como línea base para la comparación de resultados.
- Overleaf (LaTeX), para la redacción del informe de título.

- Notion, para la documentación técnica, el seguimiento de tareas mediante Kanban y el registro de avances durante el desarrollo.

5.3. Ambiente de desarrollo

El desarrollo del proyecto se realiza en dos ambientes complementarios. El primero corresponde a un equipo de trabajo personal utilizado para la implementación, pruebas unitarias y de integración del núcleo orquestador y los plugins, el cual cuenta con las siguientes características:

- Procesador AMD Ryzen 5 7600, 6 núcleos, 3.80 GHz
- Memoria RAM de 32 GB (5200 MT/s)
- Tarjeta gráfica dedicada NVIDIA GeForce RTX 5060 (8 GB VRAM) y GPU integrada AMD Radeon
- Almacenamiento SSD de 932 GB
- Sistema operativo Windows 11 Pro (versión 25H2)

En este ambiente se realiza la mayor parte del ciclo de desarrollo iterativo bajo RAD, incluyendo la implementación del orquestador, la integración de los plugins de software y las pruebas descritas en la etapa de verificación del ciclo PDCA.

El segundo ambiente corresponde a la sala de experimentación del Departamento de Ingeniería Informática de la USACH, donde se encuentra el despliegue físico de EDDIE.

Este ambiente cuenta con el computador designado para la operación de EDDIE, dos cámaras digitales utilizadas para el reconocimiento de texto y el reconocimiento gestual, un proyector actúa como display sobre el escritorio, y dispositivos de rastreo ocular como GazePoint y TheEyeTribe. Este ambiente se utiliza específicamente para la etapa de verificación del ciclo PDCA, donde se ejecutan las pruebas de aceptación y la validación del sistema con el hardware físico real, resolviendo así la dependencia de sensores virtuales documentada en etapas previas del proyecto.

6. Plan de trabajo

El proyecto se organiza en las 17 semanas y fases PDCA detalladas en la Tabla 6.1 (Anexo), consistente con el uso de prototipos desechables descrito en 5.1: las semanas 1–3 (Plan) diagnostican la arquitectura actual y validan la factibilidad técnica de migración mediante prototipos, las semanas 4–12 (Do) diseñan e implementan el núcleo, integrando primero el módulo sin dependencia de hardware (consistencia) y luego los que sí la requieren, las semanas 13–16 (Check) ejecutan las pruebas de software y verificación funcional de la sección 3.4, y la semana 17 (Act) cierra con ajustes finales y documentación técnica.

7. Glosario

- **Arquitectura modular:** Sistema de construcción que utiliza componentes prefabricados (módulos), que luego se ensamblan en el sitio final.
- **Capa de abstracción de hardware:** Componente de software que actúa como intermediario entre el sistema operativo o aplicación y el hardware físico, permitiendo interactuar con distintos dispositivos mediante una interfaz uniforme sin conocer sus detalles técnicos específicos.
- **Consistencia bidireccional:** Mecanismo de validación utilizado en diversos campos técnicos para asegurar que un proceso, dato o relación sea coherente tanto al analizarlo en una dirección como en la contraria.
- **Deuda técnica:** Costo futuro de retrabajo derivado de elegir soluciones rápidas y fáciles hoy, en lugar de un enfoque mejor diseñado pero más lento.
- **Eyetrackers:** Dispositivos tecnológicos que miden dónde mira una persona, la dirección de su mirada y el movimiento de sus ojos en tiempo real.
- **Interacción humano-información (HII):** Campo de investigación que estudia cómo las personas buscan, acceden, evalúan y usan información en distintos contextos y formatos.
- **MECIVA PS:** Metodología específica utilizada para la evaluación y validación de software científico, la cual permitió identificar los 67 errores estructurales en el sistema EDDIE actual.
- **Microservicios:** Arquitectura de software donde el sistema se divide en servicios independientes que se comunican mediante APIs, evaluada como una alternativa de solución para EDDIE por su alta escalabilidad.
- **Orquestador:** Sistema o herramienta que organiza, coordina y dispone diversos elementos para lograr un objetivo común, asegurando que funcionen de manera armoniosa.
- **Overhead de comunicación interproceso (IPC):** Tiempo y recursos adicionales de procesamiento que el sistema consume para permitir que módulos independientes intercambien información entre sí.
- **Plugins:** Aplicación informática pequeña que se integra en otro programa principal (como un navegador o gestor de contenido) para añadir funcionalidades específicas o mejorar su rendimiento sin modificar el código base.
- **Realidad aumentada:** Tecnología que superpone información digital, como imágenes, texto o datos, sobre el entorno físico real percibido por el usuario, sin reemplazarlo.

- **Science gateways:** Plataformas o portales que facilitan el acceso a recursos compartidos para la investigación científica, cuyas arquitecturas modulares sirven como referencia para la sostenibilidad de EDDIE.
- **SonarQube:** Plataforma de análisis estático de código diseñada para evaluar y mejorar continuamente la calidad, seguridad y mantenibilidad del software.
- **Visión computacional:** Área de la informática enfocada en permitir que las computadoras identifiquen y procesen objetos en imágenes o videos, utilizada en el proyecto para el reconocimiento de texto y gestos.

8. Referencias

- Bustio Martínez, L., Coma Peña, Y., & Talavera Bustamante, I. (2013). Arquitectura basada en plugins para el desarrollo de software científico. En XV Convención y Feria Internacional Informática. La Habana, Cuba.
- Carrasco Madriaga, J. P. (2017). Diseño e implementación de un sistema de realidad aumentada para el apoyo de la lectura en papel [Memoria de título, Universidad de Santiago de Chile].
- Contreras Velásquez, C. E. (2019). Desarrollo de un sistema para soportar lectura aumentada no obstrusiva [Memoria de título, Universidad de Santiago de Chile].
- Ernst, N. A., Bavota, G., & Cai, Y. (2026). The nature of technical debt in research software [Preprint]. arXiv. <https://arxiv.org/abs/2603.2041>
- Estay Vera, J. S. (2020). Integración de hardware de eye-tracking al sistema EDDIE [Memoria de título, Universidad de Santiago de Chile].
- Figueroa Contreras, M. (2025). Arquitectura basada en plugins para la captura y análisis de datos multimodales: El caso de Multimodal Observer [Memoria de título, Universidad de Santiago de Chile].
- Glatard, T., Rousseau, M. E., Camarasu-Pop, S., Adalat, R., Beck, N., Das, S., Ferreira da Silva, R., Khalili-Mahani, N., Korkhov, V., Quirion, P. O., et al. (2017). Software architectures to integrate workflow engines in science gateways. Future Generation Computer Systems, 75, 239–255. <https://doi.org/10.1016/j.future.2017.01.005>

- Ibaceta Jaña, I. A. (2023). Evaluación y validación de software científico mediante metodología MECIVA PS: Caso de estudio EDDIE [Memoria de título, Universidad de Santiago de Chile].
- Koike, H., Sato, Y., & Kobayashi, Y. (2001). Integrating paper and digital information on EnhancedDesk: A method for real-time finger tracking on an augmented desk system. *ACM Transactions on Computer-Human Interaction*, 8(4), 307–322.
<https://doi.org/10.1145/504704.504706>
- Megargel, A., Poskitt, C. M., & Shankararaman, V. (2021). *Microservices Orchestration vs. Choreography: A Decision Framework*.
- Miah, M. O., & Kong, J. (2024). Augmented reality and cross-device interaction for seamless integration of physical and digital scientific papers. *International Journal of Human–Computer Interaction*, 41(11), 7040–7057.
<https://doi.org/10.1080/10447318.2024.2388372>
- Montanares Mundaca, D. C. (2019). Desarrollo de un módulo de búsqueda web para el software EDDIE [Memoria de título, Universidad de Santiago de Chile].
- Orozco Solano, A. (2017). Diseño de Sistemas Bioinformáticos y Biología Computacional aplicados en Nanobiotecnología Molecular Médica [Tesis doctoral, Universidad Autónoma de Madrid].
- Pezoa Riquelme, I. E. (2020). Implementación de un módulo de consistencia bidireccional físico-digital para el sistema EDDIE [Memoria de título, Universidad de Santiago de Chile].
- Santibáñez Espinoza, C. A. (2020). Rediseño de interfaz de usuario para el sistema EDDIE bajo un enfoque centrado en el usuario [Memoria de título, Universidad de Santiago de Chile].
- Wander, S. (s.f.). Enlight: Augmented reading [Proyecto de grado MFA, School of Visual Arts]. Vimeo. <https://vimeo.com/93202502>
- Wellner, P. (1993). Interacting with paper on the DigitalDesk. *Communications of the ACM*, 36(7), 87–96. <https://doi.org/10.1145/159544.159630>

9. Anexo

Criterio	Enfoque 1: Corrección en C#	Enfoque 2: Microservicios	Enfoque 3: Plugins en Python
Líneas estimadas a reescribir	3.000–5.000 de 45.000 (aproximadamente 9%).	Reescritura completa de cada módulo como servicio independiente (no cuantificado).	Núcleo completo (entre 8.000–10.000 líneas resultantes).
Bibliotecas de OCR/visión disponibles	Emgu CV 4.x (binding .NET de OpenCV, 64 bits), compatible con Tesseract OCR.	Las mismas que en Enfoque 1, encapsuladas por servicio.	OpenCV nativo (Python) + pytesseract.
Errores resueltos (Tabla 3.2)	16 de 67 (infraestructura de carga y despliegue) directamente, el resto sin cambio.	Depende del diseño de cada servicio, no cuantificado.	16 de 67 directamente.
Curva de aprendizaje del equipo	Baja (stack ya conocido).	Alta (gestión de red y despliegue de servicios).	Media-alta (cambio de lenguaje).
Antecedente empírico en el laboratorio	Ninguno.	Ninguno.	MO (Figuerola Contreras, 2025), problema estructuralmente análogo.

Tabla 2.1: Comparación de sistemas.

ID	Tipo de error/problema	descripción	Estado
E11	Interno-consistencia	Función de sincronización provoca que falle el programa cuando no se ingresa un archivo .pdf.	SF
E12	Externo	No existe un manual de instalación para EDDIE.	SS
E13	Externo	No existe un manual de uso para EDDIE.	SS
E14	Interno-otro	Los mensajes de error están en un idioma distinto al resto de la aplicación.	SF

E15	Externo	No se reconoce el eyetracker en el computador.	SF
E16	Externo	Conexión de la cámara digital es inestable.	SP
E48	Interno-consistencia	En la consistencia de comentarios, ingresando correctamente los elementos necesarios para su ejecución, el programa falla.	SF
E49	Externo	Utilizando The eyetracker, no se puede detectar la parte inferior de la proyección.	SS
E50	Interno-consistencia	No se encuentra la carpeta PluginsConsistencia.	SF

Tabla 3.1: Extracto de errores encontrados en EDDIE

Fuente: (Ibaceta Jaña, 2023).

Categoría	N°	%
(a) infraestructura técnica de carga y despliegue	16	23%
(b) Hardware/sensores	9	13%
(c) Documentación	2	3%
(d) Práctica de código/lógica interna	26	39%
(e) UI/idioma	3	4%
(f) Infraestructura de laboratorio	3	4%
(g) Servicios externos/APIs	8	12%

Tabla 3.2: Conteo por categoría de los 67 errores encontrados en EDDIE

ID error original	Módulo	Estado en EDDIE (C#)	Categoría (Tabla 3.1)	Reproducido en prototipo Python	Resultado en nueva arquitectura	Evidencia
E38	Consistencia	SF	(a)	Sí	Corregido (eliminado por diseño)	Test unitario test_plugin_loader_missing_file
E51	Rastreo ocular	SF	(a)	Sí	Corregido (eliminado por diseño)	Test unitario test_plugin_registry_eyetracker
E40	Consistencia	SS	(d)	Sí	Persiste (requiere recalibración de OpenCV)	Log de prueba de aceptación PA-XX
E63	Búsqueda	SP	(g)	No aplica	Fuera de alcance (depende de API externa expirada)	Nota de alcance, sección 3.3

Tabla 3.3: Ejemplo de matriz de trazabilidad de los errores de EDDIE

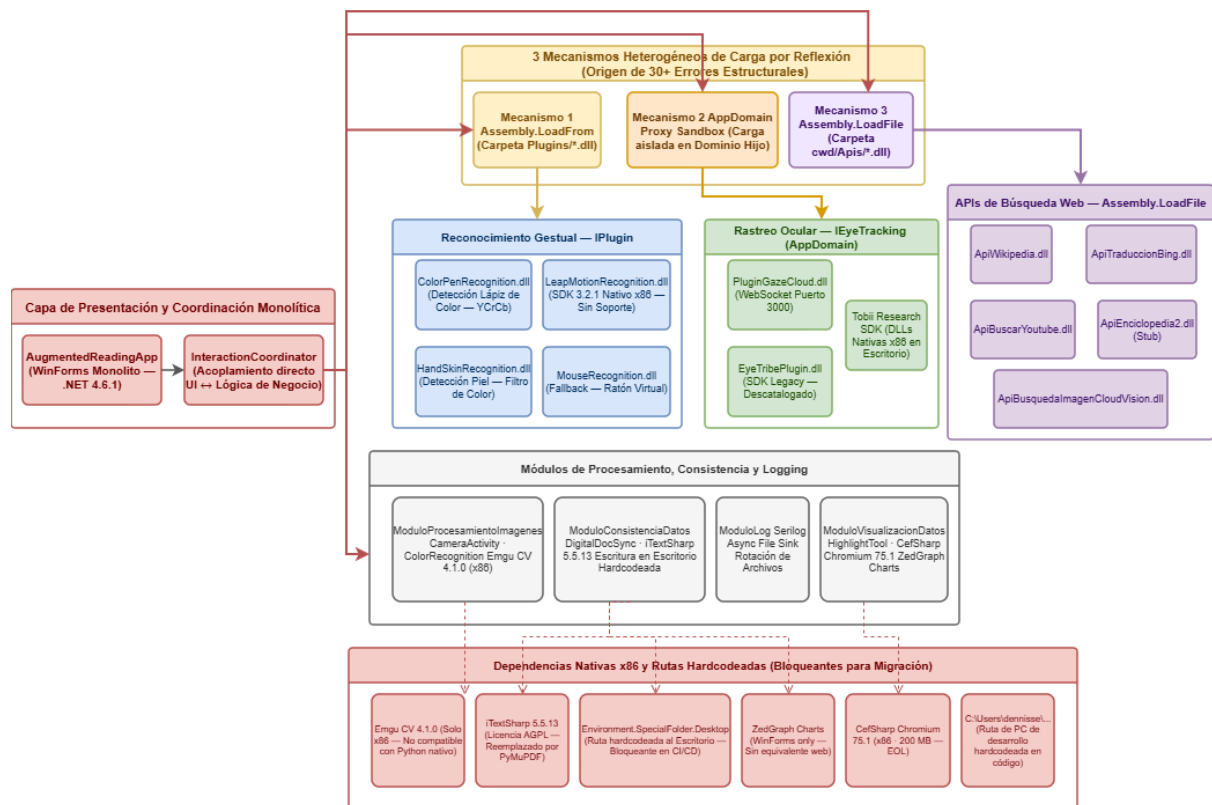


Figura 3.1: Arquitectura original monolítica de EDDIE

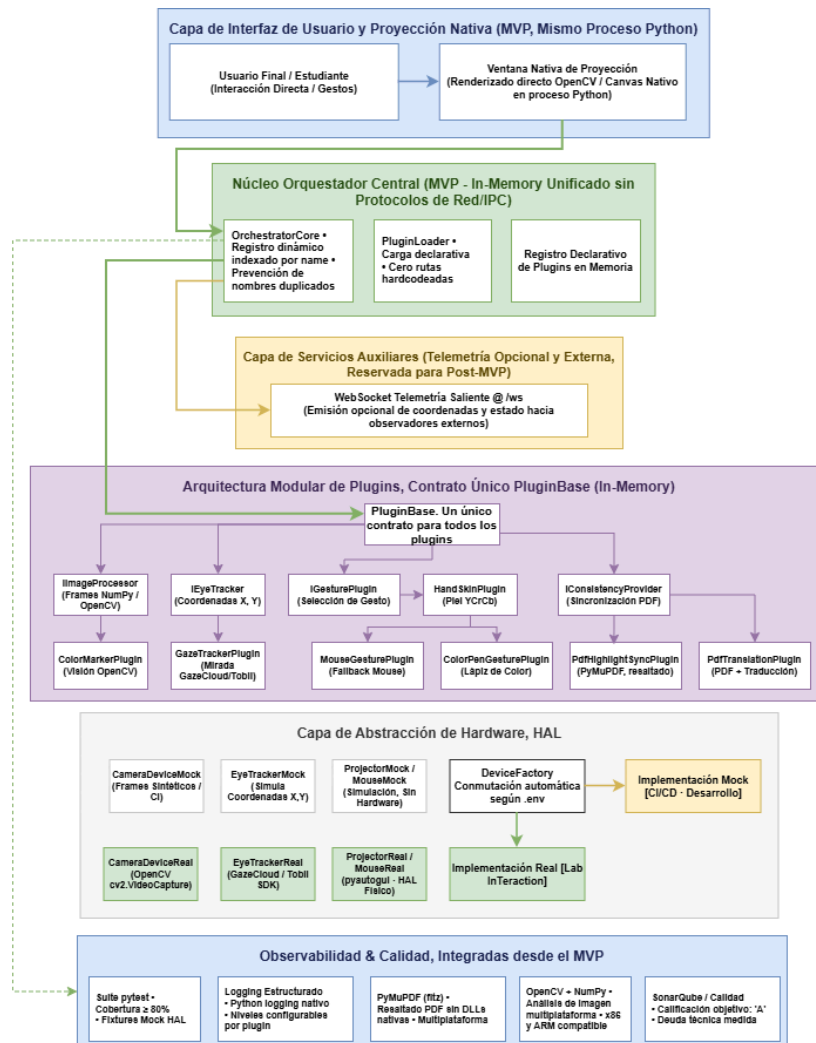


Figura 3.2: Arquitectura objetivo modular de EDDIE

Semanas	Fase PDCA	Actividades	Objetivo específico	Horas estimadas	Entregable
1-3	Plan	Diagnóstico de la arquitectura actual de EDDIE en C# mediante reclasificación de errores (Tabla 3.2), y construcción de prototipos desechables en Python para validar la factibilidad técnica de migrar cada módulo del MVP a las tecnologías propuestas (OpenCV, PyMuPDF, SDKs de eyetrackers).	OE1	108 h	Documento de diagnóstico con línea base reclasificada y resultados de prototipos de factibilidad en Python
4-6	Do (Diseño)	Diseño del núcleo orquestador en Python y definición de los contratos de interfaz para la integración de plugins, en base a los resultados de los prototipos desechables de la fase anterior. Primer prototipo funcional bajo RAD.	OE2	108 h	Diagrama de arquitectura y contratos de interfaz
7-10	Do (Implementación)	Implementación del núcleo orquestador y de la capa de abstracción de sensores. Integración iterativa del módulo de consistencia como plugin, y desarrollo inicial de los plugins de reconocimiento de texto (IImageProcessor) e interacción gestual (IGesturePlugin) sobre las cámaras digitales del laboratorio.	OE3	144 h	Prototipos funcionales del núcleo con módulos integrados
11-12	Do (Implementación)	Integración del módulo de rastreo ocular con hardware físico (GazePoint/TheEyeTribe) y ajustes de la capa de abstracción según resultados de pruebas tempranas. Contingencia: si ninguno de los dos eyetrackers logra integrarse con su SDK en Python dentro del plazo, el módulo se valida únicamente mediante EyeTrackerMock (DeviceFactory), documentando la limitación sin bloquear la entrega de los otros dos módulos del MVP.	OE3	72 h	Prototipo integrado con hardware físico (o con mock documentado, según contingencia)

13-14	Check	Pruebas de software: cobertura de líneas de código, matriz de trazabilidad, latencia orquestador-plugin y métricas de sincronización/visión computacional.	OE4	72 h	Reporte de pruebas de software
15-16	Check	Verificación y validación: aplicación de MECIVA PS por módulo (requisitos críticos/secundarios), pruebas de validez y confiabilidad, y pruebas de aceptación en el ambiente operativo real del laboratorio.	OE4	72 h	Reporte de verificación y validación
17	Act	Incorporación de ajustes finales derivados de las pruebas, cierre de la documentación técnica y entrega del informe final.	OE4	36 h	Documentación técnica y entrega final

Tabla 6.1: Plan de trabajo.